CS 4390 Project 1 Report (Centralized Math Server)

a. Names and Net ID:

i. Nguyen Do, npd220001

ii. Jeremiah Boban, jxb220076

b. Partner Contribution

i. Nguyen Do

1. Designed TCPServer's logic to process client's request, solve mathematical calculation using the Shunting Yard algorithm and convert equations to postfix notation to solve them. The server also logged the client's activities in a separate file.
2. Designed TCPClient's logic to create clients, tracking their connection status, and multiple client connections.

ii. Jeremiah Boban

1. Implemented a producer-consumer architecture using a BlockingQueue and a RequestProcessor thread. This ensures the server handles math requests in the exact order they are received (FIFO) across all clients. Even if multiple clients simultaneously send their requests, the server can process them in the order they hit the buffer.
2. Refined the communication protocol to use a specific command-based structure and created a Makefile to automate the build and execution process.

c. Protocol Design

i. A stateful TCP protocol to track client's connection state via the server after the 3-way handshake. For every request, the client and server communicate with each other by parsing the equations into tokens that will be used for Shunting Yard and be solved. Finally, the client can choose to exit after every set of 3 requests or at any time as they chose, and the

server tracks the time and duration of the connection with said client. The server is also staying indefinitely for more clients, but you can manually exit the program to close its connection. We also added a BlockingQueue which orders requests in a First in First out manner, avoiding any thread related conflicts.

ii. Assumptions: Equations when typed, there must be a space between 2 tokens (Ex: 1 + 4 * 12)

iii. Message format for sending and receiving math calculations:

1. Request: Encoded string containing operators and operands, separated by a single space (Ex: 1 + 4 * 12 \n), where \n is caused by pressing the Enter key, which tells the server that the message is completed.
2. Response: Encoded string with unique status prefixes.
    a. Success: Result: <value> \n (Ex: Result: 49 \n)
    b. Failure: Error: <description> \n (Ex: Error: Unknown token 'a' \n)

iv. Message format for joining and terminating the connection.

1. Joining: The client types in their name as a string, followed by \n.
    a. The server responds with an ACK string: Welcome client <name>, you joined at time: yyyy/MM/dd HH:mm:ss \n
2. Terminating: When the client encounters the message: Do you like to stay for 3 more equations? (YES/NO), they can type NO to terminate the connection.
    a. The client will see the message: "Connection closed at yyyy/MM/dd HH:mm:ss. Duration: <value> s"

v. Format for keeping logs of clients' activities on the server side

1. The server uses a log format to keep track of client's activities and requests
    a. Connection: Client <name> connected at yyyy/MM/dd HH:mm:ss"
    b. Request: Request from client <name>: <equation>

c. Disconnection: Client <name> disconnected at yyyy/MM/dd HH:mm:ss. Duration <value>s

2. In a separate log file, the server can also keep track of client activities in a formal manner:
    a. Connection: [yyyy/MM/dd HH:mm:ss] JOIN: <client name>
    b. Request: [yyyy/MM/dd HH:mm:ss] REQUEST from <client name>: <request>
    c. Disconnection: [yyyy/MM/dd HH:mm:ss] QUIT: <client name>. Duration: <value> s

d. Programming Environment

i. We used Visual Studio Code to program this project. Powershell was used in multiple terminals, acting as a server and multiple clients to compile and run the project.

e. How to compile and execute your programs.

i. Make sure you are in the project directory. A Makefile is provided, so just run **make**. If **make** doesn't work, it might be due to our Makefile formatting. In that case, use **make run-server** and **make run-client**. Create new terminals for each client you want to connect.

ii. Another option is locate the project directory, and run javac TCPServer.java TCPClient.java. Then designate one terminal to be a server and run java TCPServer. Designate 1+ terminals and run java TCPClient.

f. Parameters needed for execution

i. IP Address: 127.0.0.1, a localhost to test in a single machine (Hardcoded)

ii. Port: 6789 (Hardcoded)

iii. Inputs: Client's names and any math expressions (separated by spaces)

g. Good comments

i. Comments are placed in each section of the code to explain in detail what they do.

h. Application completed?

i. The application is completed within the specified time frame.

i. Challenges faced?

i. Sometimes clients can interfere with each other, since they are independent threads, the stacks used for the Shunting Yard algorithm cannot be global as expressions can be mixed up. I made them local so that it is unique for each client.

ii. The actual Shunting Yard algorithm is somewhat difficult to implement because the order operations must be respected to get results. By setting each operator its own precedence, this algorithm can process equations smoothly and produce accurate solutions.

iii. There were some issues logging the client's connection arrival and departure times, so I switched to LocalDateTime, an all in one time tracker that can gather the dates and specific times that the clients join.

iv. Handling multiple clients require new sockets, which is an additional feature that we implemented on top of the given base code.

v. Handling asynchronous client requests in a synchronous processing order was a challenge, which we solved using a queue dedicated for requests.

j. What have you learned doing this project?

i. Socket programming is something I am not familiar with to begin with, and this project gives me confidence in building a simple yet robust network application that manages the TCP connection life cycle.

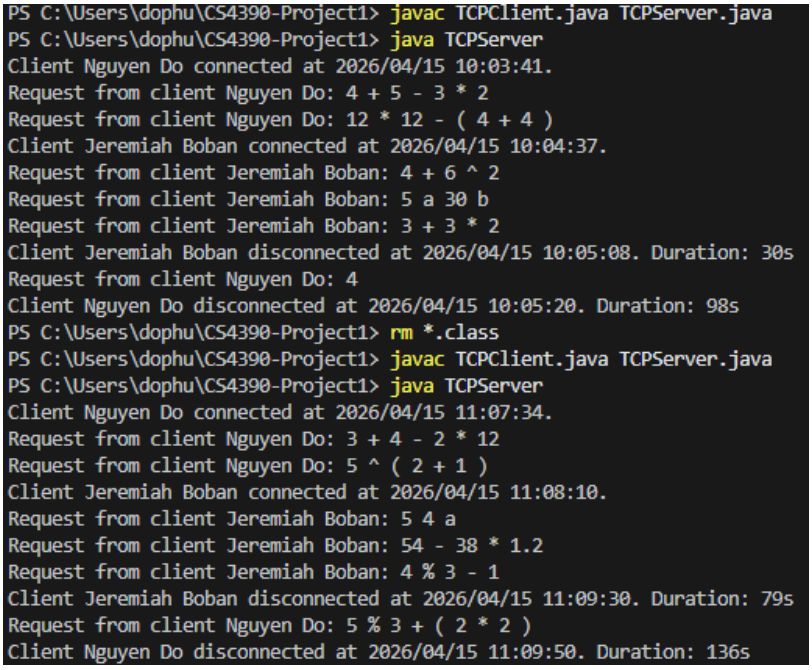
ii. I learned that the server must stay responsive to process clients that can join at any time. Multi-threading is another consideration that adds flexibility to the server's responsiveness.

iii. Specific error handling is important, not just general handling to TCP connection errors but also bad token or improper client inputs, as this can lead to bad results.

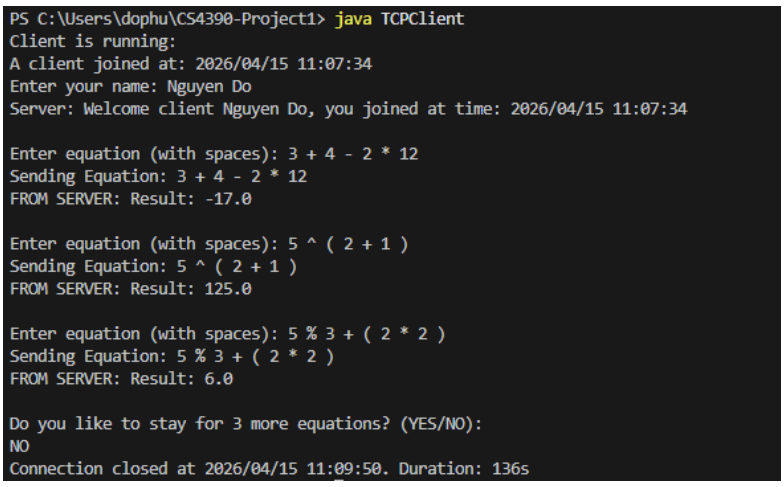
iv. A queue is necessary to enforce a FIFO order for threads. While testing, we overlooked this flaw because our project worked well at first glance, but queueing requests is necessary in a large-scale network.

k. Project output screenshots:

i. Server output:



ii. Client 1 output:



iii. Client 2 output:

```
PS C:\Users\dophu\CS4390-Project1> java TCPClient
Client is running:
A client joined at: 2026/04/15 11:08:10
Enter your name: Jeremiah Boban
Server: Welcome client Jeremiah Boban, you joined at time: 2026/04/15 11:08:10

Enter equation (with spaces): 5 4 a
Sending Equation: 5 4 a
FROM SERVER: Error: Unknown token 'a'

Enter equation (with spaces): 54 - 38 * 1.2
Sending Equation: 54 - 38 * 1.2
FROM SERVER: Result: 8.399999999999999

Enter equation (with spaces): 4 % 3 - 1
Sending Equation: 4 % 3 - 1
FROM SERVER: Result: 0.0

Do you like to stay for 3 more equations? (YES/NO):
NO
Connection closed at 2026/04/15 11:09:30. Duration: 79s
```

iv. Log output (sample trace for 2 runs):

```
1  [2026/04/15 10:03:41] JOIN: Nguyen Do
2  [2026/04/15 10:04:00] REQUEST from Nguyen Do: 4 + 5 - 3 * 2
3  [2026/04/15 10:04:15] REQUEST from Nguyen Do: 12 * 12 - ( 4 + 4 )
4  [2026/04/15 10:04:37] JOIN: Jeremiah Boban
5  [2026/04/15 10:04:47] REQUEST from Jeremiah Boban: 4 + 6 ^ 2
6  [2026/04/15 10:04:54] REQUEST from Jeremiah Boban: 5 a 30 b
7  [2026/04/15 10:05:01] REQUEST from Jeremiah Boban: 3 + 3 * 2
8  [2026/04/15 10:05:08] QUIT: Jeremiah Boban. Duration: 30s
9  [2026/04/15 10:05:17] REQUEST from Nguyen Do: 4
10 [2026/04/15 10:05:20] QUIT: Nguyen Do. Duration: 98s
11 [2026/04/15 11:07:34] JOIN: Nguyen Do
12 [2026/04/15 11:07:50] REQUEST from Nguyen Do: 3 + 4 - 2 * 12
13 [2026/04/15 11:08:04] REQUEST from Nguyen Do: 5 ^ ( 2 + 1 )
14 [2026/04/15 11:08:10] JOIN: Jeremiah Boban
15 [2026/04/15 11:08:20] REQUEST from Jeremiah Boban: 5 4 a
16 [2026/04/15 11:08:30] REQUEST from Jeremiah Boban: 54 - 38 * 1.2
17 [2026/04/15 11:09:25] REQUEST from Jeremiah Boban: 4 % 3 - 1
18 [2026/04/15 11:09:30] QUIT: Jeremiah Boban. Duration: 79s
19 [2026/04/15 11:09:45] REQUEST from Nguyen Do: 5 % 3 + ( 2 * 2 )
20 [2026/04/15 11:09:50] QUIT: Nguyen Do. Duration: 136s
21
```